P

PayBill

On-Chain Utility Payments & Recurring Schedules on BOT Chain

WHITEPAPER | VERSION 1.0
October 2026 · BOT Chain · Chain ID 968

PROTOCOL OVERVIEW

PayBill is a utility and bill payment protocol on BOT Chain that generates immutable, on-chain receipts for every payment. Admin-registered billers receive payments directly; payers supply an account reference string that is stored alongside each receipt for reconciliation. PayBill also supports prepaid recurring payment schedules: a payer deposits the full amount upfront, and individual payments release to the biller automatically when each due date arrives. Anyone can trigger a due payment, removing the need for a centralised scheduler.

A technical overview of the protocol, its design, core functions, security properties, and roadmap.

1. Problem Statement

• Unverifiable receipts — paper or PDF receipts can be forged; there is no authoritative on-chain record.

• Missed recurring payments — bank standing orders fail silently; late fees accumulate.

• No standard biller registry — each utility has its own payment portal with different account formats.

• No refund guarantee — if a recurring payment is set up incorrectly the payer has no automatic recourse.

2. Protocol Design

2.1 One-Off Payment Flow

• Admin registers a biller with a name, category, and wallet address.

• User calls pay(billerId, ref) with the exact ETH amount.

• ETH is immediately forwarded to the biller's wallet.

• A Receipt is stored on-chain — payer, biller, ref, amount, timestamp.

2.2 Recurring Schedule Flow

```
Step
Action
```

1. Create

createSchedule(billerId, ref, intervalDays, count) — deposit count * amount upfront

2. Release

runSchedule(scheduleId) — callable by anyone when block.timestamp >= nextDue

3. Cancel

cancelSchedule(scheduleId) — payer cancels; remaining deposits refunded

The deposit must be exactly divisible by count, guaranteeing equal per-payment amounts. Intervals range from 1 to 365 days; maximum 24 payments per schedule.

3. Contract Architecture

3.1 Data Structs

Biller: name, category, wallet address, active flag.

Receipt: payer, billerId, ref string, amount, timestamp.

Schedule: payer, billerId, ref, amount per payment, interval (seconds), remaining count, nextDue timestamp, active flag.

3.2 Write Functions

```
Function
Role
```

Description

```
addBiller(name, category, wallet)
```

	Admin	
	Registers a biller	
setBillerActive(id, active)		
	Admin	
	Toggles biller on/off	
`transferAdmin(addr)`		
	Admin	
	Transfers admin role	
pay(billerId, ref)		
	User	
	One-off payment with reference	
createSchedule(bid, ref, days, count)		
	User	
	Creates prepaid recurring schedule	
runSchedule(scheduleId)		
	Anyone	
	Releases next due payment	
cancelSchedule(scheduleId)		
	Schedule Payer	
	Cancels schedule and claims refund	

4. Security Properties

4.1 Reentrancy Guard

Both pay() and runSchedule() are protected by a nonReentrant modifier using a boolean lock — ETH is forwarded only after state is updated.

4.2 Even Split Invariant

require(msg.value % count == 0) enforces exact divisibility at schedule creation, ensuring no remainder is stranded in the contract.

4.3 Biller Availability Check

runSchedule() checks billers[s.biller].active before each payment release — a deactivated biller blocks future schedule payments, protecting funds.

4.4 Direct Forwarding

Biller funds are never pooled in the contract for one-off payments — ETH flows directly from payer to biller wallet in the same transaction.

5. Events

Event	Parameters
BillerAdded	

id, name, wallet

BillerStatus

id, active

Paid

receiptId, payer, billerId, amount

ScheduleCreated

id, payer, billerId

ScheduleCancelled

id

6. Deployment

Parameter	Value
Chain	BOT Chain (Chain ID 968)
RPC	https://rpc.botchain.ai
Schedule limits	1–365 day interval, 1–24 payments, exact-split deposit

7. Roadmap

Milestone	Description
v1.1	ERC-20 stablecoin support for bill payments
v1.2	Automatic runSchedule via Keeper / Gelato network integration
v1.3	Payment splitting — split a bill among multiple payers
v2.0	Biller self-registration with admin approval queue